



MedOrbit

Smart Healthcare Platform

Ali Ghassan Abdullah Dawood · Omar Abdullah Sadeq Abu Mazen

Supervisor: Dr. Bashar Tahyana

Computer Engineering · An-Najah National University · 2026

Healthcare access is fragmented

1

Scattered discovery.

Finding a trusted doctor or clinic is spread across disconnected sources.

2

Disconnected records.

Appointments, medical records and prescriptions live in separate places.

3

Limited guidance.

Patients often don't know which specialty fits their symptoms.

4

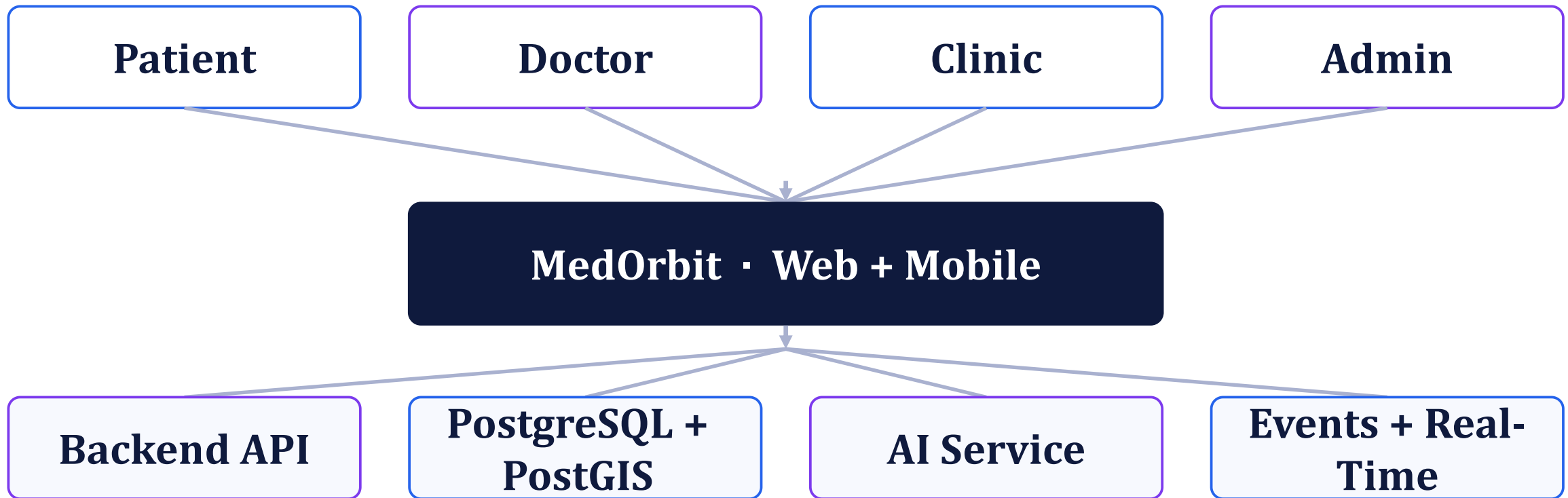
Siloed roles.

Patients, doctors, clinics and administrators use unrelated tools.

THE SOLUTION

One unified healthcare platform

Every role, both clients and all intelligence on a single shared backend.



Who uses MedOrbit?

Five distinct roles, each with its own interface and its own authorization rules.



Patient

- Discover care
- Manage health data
- AI health tools



Doctor

- Patients & schedule
- Records & prescriptions
- Health posts



Clinic

- Facility workspace
- Invite doctors
- Manage services



Admin

- Users & applications
- Content moderation
- Audit trail



SuperAdmin

- Highest privileges
- Invite admins

01

Platform Features

What each role can actually do on the platform today.



Patient experience

1

Discover care.

Search doctors by specialty and find clinics on an interactive map.

2

Book appointments.

See real availability and reserve a slot in a guided flow.

3

Manage health data.

Medical records, prescriptions and personal reports in one place.

4

Stay connected.

Message your doctor, follow health posts and receive notifications.

Doctor workspace

1

Professional profile.

Specialty, experience, expertise and clinic affiliation.

2

Schedule & appointments.

Define availability rules, then confirm and complete visits.

3

Patient care.

Assigned patients, medical records and clinical notes.

4

Prescriptions.

Issue prescriptions with AI-assisted interaction warnings.

CORE FEATURES

Clinic workspace

A clinic is a first-class account type, not just a record inside the directory.

- 1 Apply and get verified.**
A facility applies, an administrator reviews and approves it.
- 2 Manage the facility.**
Services, location, contact details and verified contact emails.
- 3 Invite doctors.**
Doctors join a clinic only by accepting an invitation — never by assignment.
- 4 Controlled credentials.**
Registration numbers change only through an admin-reviewed request.

Administration

1

Platform analytics.

Usage across users, specialties, appointments and activity.

2

Application review.

Approve or reject doctor and clinic applications.

3

Users & moderation.

Manage accounts and moderate the public health feed.

4

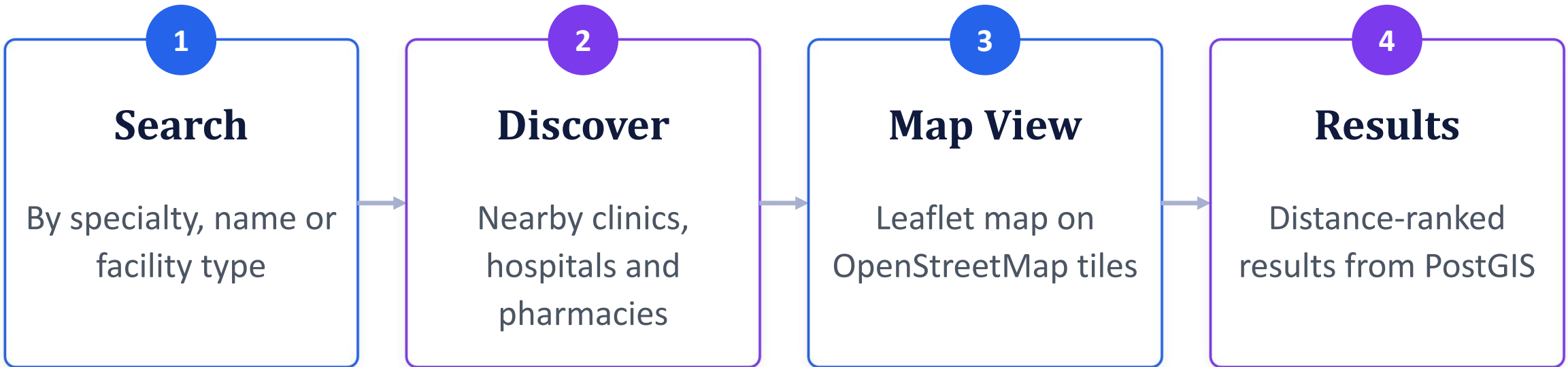
Audit & invitations.

Every sensitive action is logged; super admins invite new admins.

CORE FEATURES

Healthcare discovery & map

Finding the right doctor or facility, visually.



Leaflet + OpenStreetMap on both clients; PostgreSQL/PostGIS distance queries on the server.

Appointments

Two sides of one booking flow, guarded by the database.

Patient

- Select a doctor
- View real available slots
- Choose a time and book
- Track, confirm or cancel

Doctor

- Define availability rules
- Block days and time windows
- Confirm and complete visits
- A booked slot can never be double-taken

Medical records & prescriptions

Clinical data with relationship-based access.

Patient

- View medical record timeline
- Open full record details
- View and download prescriptions

Doctor

- Access records of assigned patients only
- Write clinical notes
- Create prescriptions with safety checks

Two different conversations

Human messaging and AI assistance are deliberately separate systems.

Patient ↔ Doctor · Socket.IO

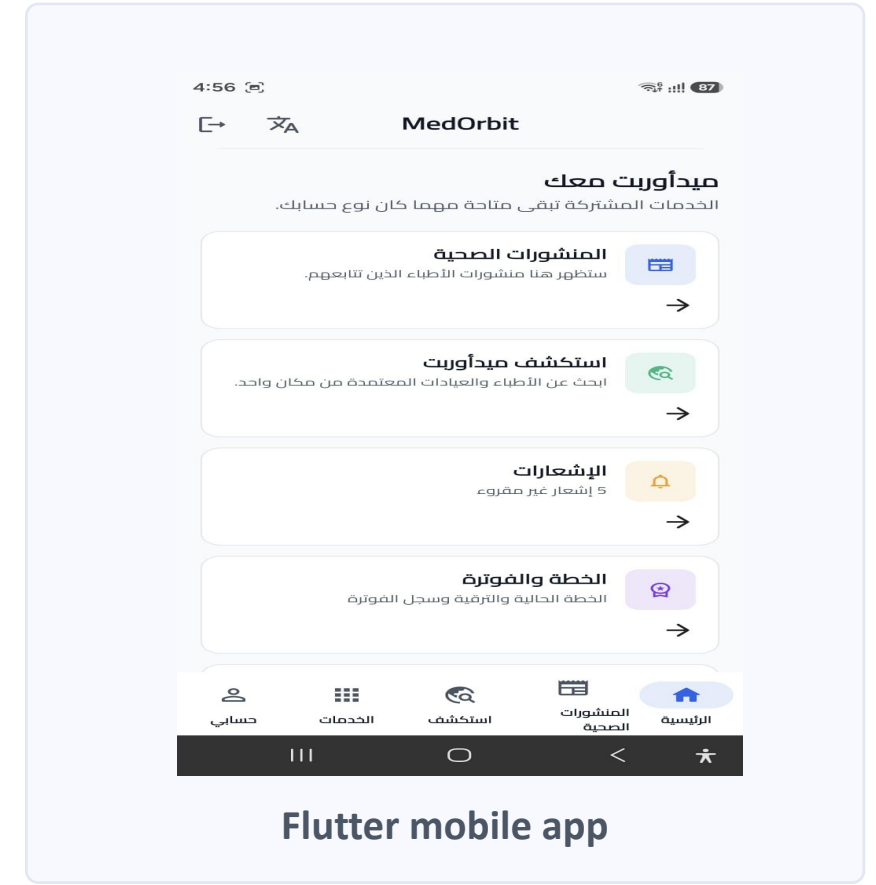
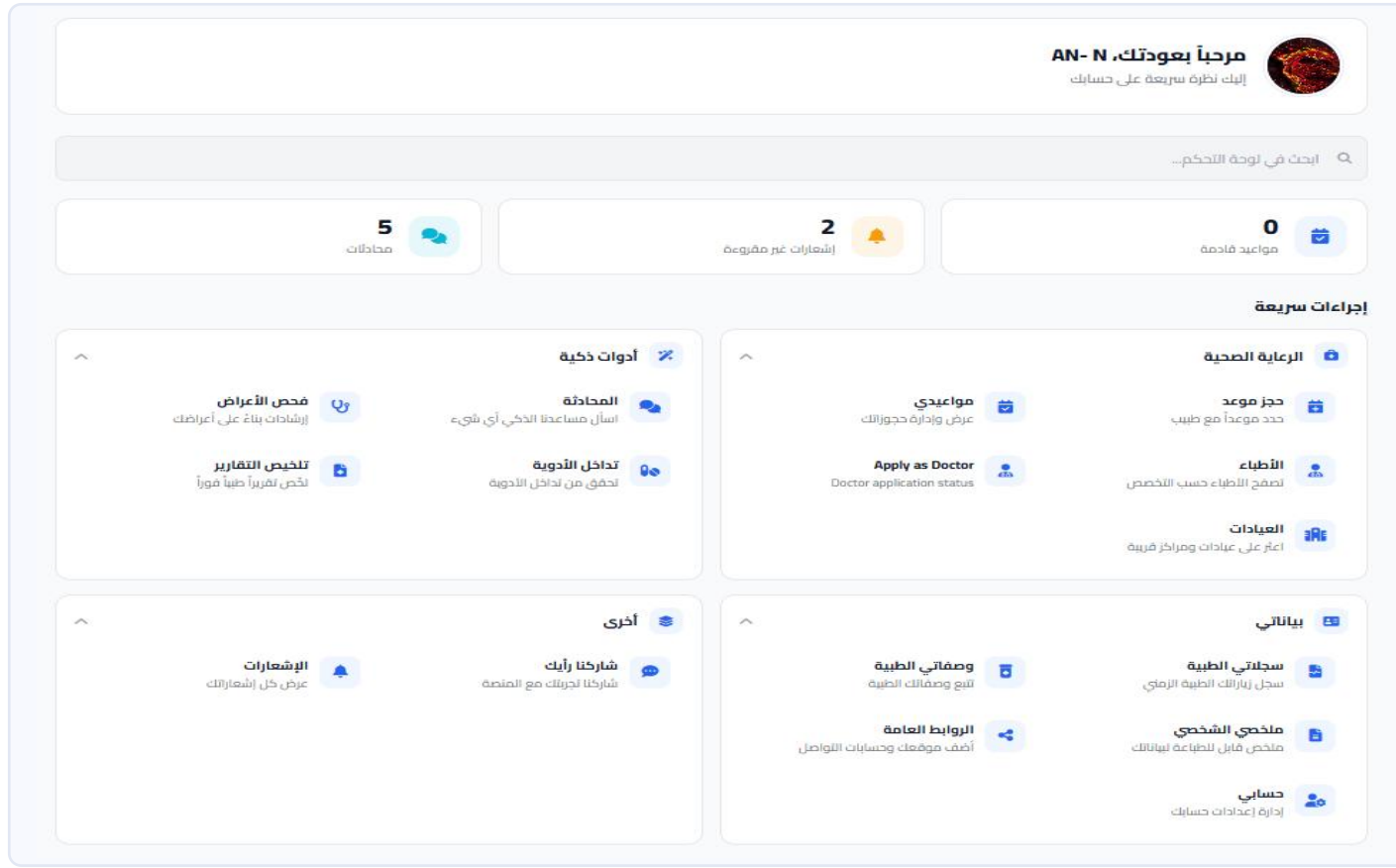
- Real-time chat over an authenticated socket
- A real human doctor replies
- PostgreSQL stays the source of truth
- Patients control who may message them

AI Assistant · AI Service

- Handled entirely by the AI service
- Language understanding, safety layer, local LLM
- Automated — never a substitute for a doctor
- Kept fully separate from human conversations

One account, two clients

The same live data on the web application and the Flutter mobile app.



02

Artificial Intelligence

Five assistance features, and the safety layers underneath them.



What the AI actually does

1

Symptom Triage

- Rule-based specialty matching
- Confidence score, not a diagnosis

2

Medical Chatbot

- Arabic & English understanding
- Local LLM via Ollama

3

Drug Checker

- Database-backed interactions
- Severity and Rx warnings

4

Report Summarizer

- OCR for scans and PDFs
- Arabic & English summaries

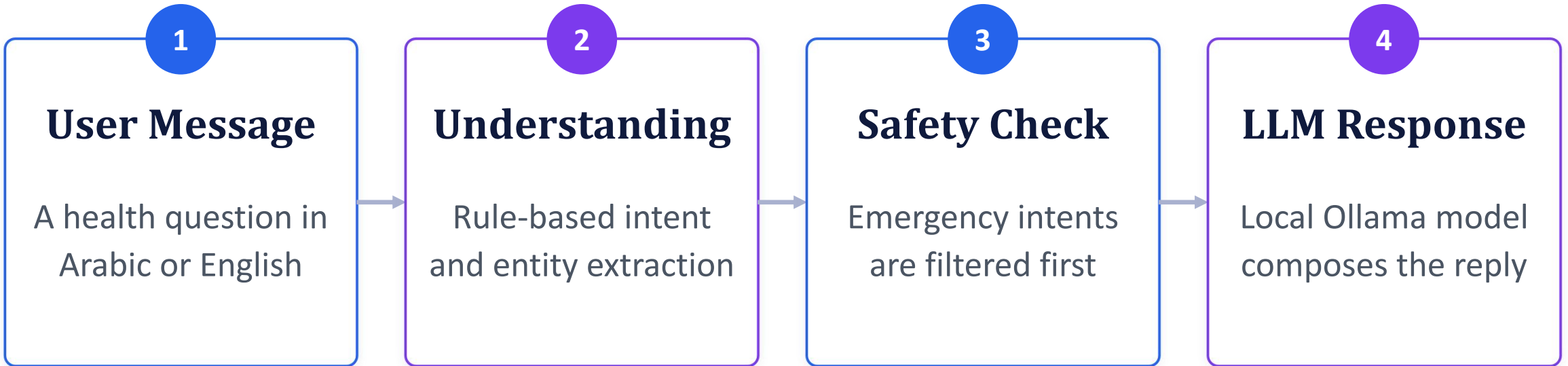
5

Virtual Doctor

- Guided voice consultation
- Grounded, safety-checked

How the medical chatbot works

From a patient message to a safe, grounded reply.



The backend authenticates every request — the AI service is internal-only and never exposed to clients.

How the Virtual Doctor works

A structured voice consultation — our deepest AI subsystem.



Session memory lives in the database, so the interview builds on every previous answer.

How we keep the AI safe

The most important property of a medical assistant is knowing when to stop and escalate.



Emergency First

- Emergency patterns are checked before anything else
- Arabic and English



Symbolic Rules

- A Prolog rule engine encodes clinical red flags
- Deterministic, not probabilistic



Escalate Only

- Urgency can be raised, never lowered
- Enforced in four separate places



Clear Limits

- Guidance, never a diagnosis
- Every AI output is advisory

03

Engineering

The architecture, the data, and the discipline behind the product.



System architecture

A service-oriented architecture: every part has one job and one boundary.

1

Clients

- Web: HTML/CSS/JS + nginx
- Mobile: Flutter + Dart
- One shared REST contract

2

Backend

- Node.js / Express API
- Authentication + access control
- Socket.IO real-time layer

3

Data

- PostgreSQL 18 + PostGIS
- Hand-written SQL, no ORM
- 31 ordered migrations

4

AI + Events

- Python / FastAPI, internal-only
- Local LLM via Ollama
- Kafka + background workers

Database design

PostgreSQL 18 + PostGIS · geospatial search · JSONB where it earns its place · 31 tracked migrations.

1

Core Healthcare

- Users, doctors & specialties
- Clinics & facilities
- Appointments
- Records & prescriptions

2

Communication

- Conversations & messages
- Notifications
- Health feed & follows

3

Platform & AI

- Subscriptions & usage
- Audit logs & event outbox
- AI and consultation sessions

Asynchronous events & real-time

Two different problems solved with two different patterns.

Kafka · Asynchronous

- Events are saved in the same transaction as the data
- A worker publishes them with retry and back-off
- Consumers build recommendation profiles
- The user's request never waits for Kafka

Socket.IO · Real-Time

- Authenticated live connections
- Room-based patient ↔ doctor messaging
- Delivered instantly, stored in PostgreSQL
- Independent from the AI assistant

Authentication & security

Healthcare data needs layered boundaries — not just a login screen.

1

Identity

- Email/password + Google Sign-In
- Access & refresh tokens
- Passwords hashed with bcrypt

2

Authorization

- Role-based access for 5 roles
- Doctor access follows the care relationship
- Changing a role revokes sessions instantly

3

Platform Defense

- Security headers & content policy
- CORS rules & rate limiting
- Audit trail for sensitive actions

Delivery & verification

Reproducible environments and automated checks on every push.

Deployment

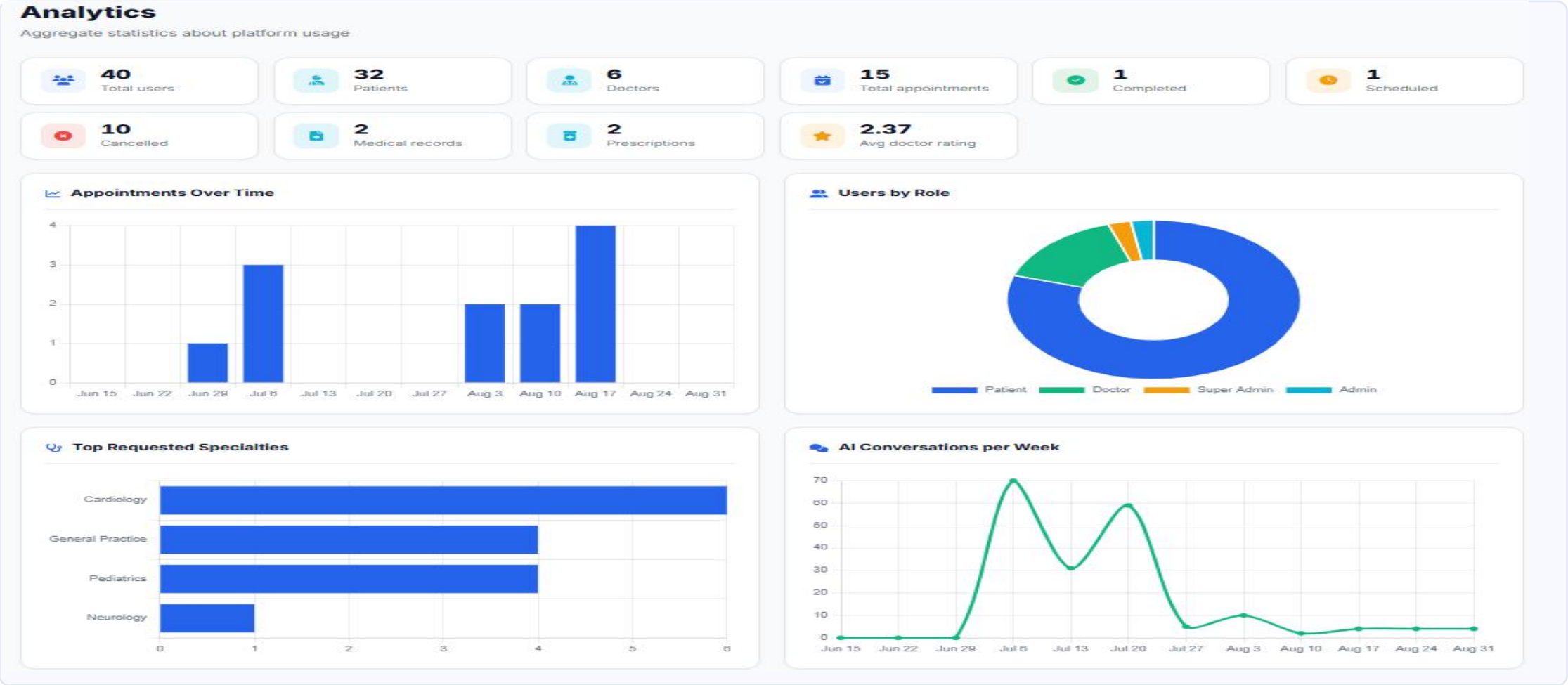
- Docker Compose runs the whole stack
- Backend, AI service, database, Kafka, workers, web
- Health checks before traffic is served
- HTTPS staging configuration prepared

Testing & CI

- Integration tests: auth, access control, messaging
- Event pipeline and billing test suites
- AI service unit and safety-regression tests
- GitHub Actions: build, migrate, test, health-check

RESULTS

Platform analytics dashboard



What we actually delivered

One platform, engineered across every layer.

✓ **Healthcare Discovery**

✓ **Appointments**

✓ **Medical Records**

✓ **Prescriptions**

✓ **Patient ↔ Doctor
Messaging**

✓ **Clinic Workspace**

✓ **Administration & Audit**

✓ **AI Assistance (5
tools)**

✓ **Maps & Geospatial
Search**

✓ **Real-Time
Communication**

✓ **Event-Driven
Processing**

✓ **Web + Mobile Clients**

Limitations & future work

An honest picture of where the prototype stands today.

Current Limitations

- Academic prototype, not a certified medical device
- AI output requires professional review
- Payment is simulated, not a live provider
- Analytics load on request, not in real time

Future Work

- Physical-device and load testing
- Independent security review
- Real payment-gateway integration
- Production monitoring and backups

Languages & technologies

Every language and platform behind MedOrbit, and where each one is used.

JS

JavaScript

Web application —
HTML5, CSS3, nginx

Dart

Flutter

Mobile app — Riverpod,
GoRouter, Dio

Node

Node.js

Express REST API +
Socket.IO

Python

FastAPI

AI service — NLU, RAG,
OCR, voice

SQL

PostgreSQL

PostGIS geospatial data,
31 migrations

Prolog

SWI-Prolog

Symbolic clinical
reasoning rules

Kafka

Apache Kafka

Asynchronous event
pipeline

Docker

Docker

Compose stack for every
service



Thank You

Questions & Discussion

Ali Ghassan Abdullah Dawood · Omar Abdullah Sadeq Abu Mazen